Universidad del Valle de Guatemala

Facultad de Ingeniería

Catedrático: Kevin Antonio Velásquez Aguilar

Programación Orientada a Objetos

Laboratorio 2 – Agencia de Detectives

Nombre: María Isabel Cajas Cuéllar

Carné: 26183

Sección: 20

Fecha de entrega: 30/09/2026

Análisis del Sistema

1.1 Requisitos funcionales del sistema

- Solicitar el nombre del caso, código de identificación y detective responsable al iniciar el programa.
- Permitir crear un nuevo caso que reemplace al actual, sin ubicaciones ni pistas registradas.
- Almacenar un máximo de cinco ubicaciones mediante un arreglo básico de objetos “Ubicación”.
- Registrar cada ubicación con código, nombre, dirección, nivel de riesgo y estado, en la posición indicada por el usuario.
- Validar que la posición esté dentro de los límites del arreglo y se encuentre disponible.
- Validar que el nivel de riesgo esté entre 1 y 10 al crear o modificar una ubicación.
- Verificar las posiciones con “null” antes de utilizar una ubicación.
- Consultar todas las ubicaciones registradas y sus posiciones.
- Consultar una ubicación específica por su posición e informar si está vacía o es inválida.
- Modificar el nivel de riesgo y el estado de una ubicación registrada.
- Descartar una ubicación y dejar su posición en “null”.
- Almacenar las pistas utilizando directamente “ArrayList<Pista>”, sin utilizar List.
- Registrar cada pista con código, descripción, tipo de evidencia, nivel de importancia y nivel de confiabilidad.
- Validar que la importancia esté entre 1 y 10 y la confiabilidad entre 0 y 100.
- Impedir el registro de pistas con códigos repetidos.
- Consultar todas las pistas e informar cuando no existan registros.
- Buscar una pista por su código recorriendo el “ArrayList”.
- Modificar la información de una pista registrada, aplicando las mismas validaciones de su creación.
- Eliminar una pista del “ArrayList” mediante su código.
- Mostrar la cantidad de ubicaciones registradas y los espacios disponibles.
- Mostrar la ubicación con mayor nivel de riesgo.
- Mostrar la cantidad de pistas registradas.
- Mostrar la pista con mayor importancia y la de mayor confiabilidad.
- Calcular el promedio de importancia de las pistas registradas.

- Evitar operaciones que requieran elementos cuando el “ArrayList” esté vacío.
- Manejar entradas numéricas incorrectas mediante “InputMismatchException” y limpiar la entrada.
- Generar “IllegalArgumentException” cuando los niveles no cumplan los rangos establecidos.
- Utilizar try-catch para manejar errores y permitir que el programa continúe funcionando.
- Utilizar al menos un bloque “finally” para ejecutar una acción final, haya ocurrido un error o no.
- Mostrar el menú de las trece opciones y finalizar cuando el usuario seleccione “Salir”.

1.2 Atributos de cada clase

Clase: Caso

Atributo	Tipo de dato	Visibilidad	Propósito
nombre	String	private	Guarda el nombre del caso.
codigoIdentificacion	String	private	Identifica el caso.
detectiveResponsable	String	private	Guarda el nombre del detective encargado.
ubicaciones	Ubicacion[]	private	Almacena hasta cinco ubicaciones; los espacios vacíos contienen null.
pistas	ArrayList<Pista>	private	Almacena una cantidad dinámica de pistas.

Clase: Ubicacion

Atributo	Tipo de dato	Visibilidad	Propósito
codigo	String	private	Identifica la ubicación.
nombre	String	private	Guarda el nombre del lugar.
direccion	String	private	Indica dónde se encuentra el lugar.

nivelRiesgo	int	private	Representa el riesgo, entre 1 y 10.
estado	String	private	Describe el estado de la ubicación.

Clase: Pista

Atributo	Tipo de dato	Visibilidad	Propósito
codigo	String	private	Identifica la pista y permite buscarla.
descripcion	String	private	Describe la pista encontrada.
tipoEvidencia	String	private	Indica el tipo de evidencia.
nivelImportancia	int	private	Representa la importancia, entre 1 y 10.
nivelConfiabilidad	int	private	Representa la confiabilidad, entre 0 y 100.

Clase App o Main: No tiene atributos

1.3 Métodos de cada clase

Clase: Caso

Método	Parámetros	Retorno	Propósito
Caso	String nombre, String codigoIdentificacion, String detectiveResponsable	No aplica	Inicializa los datos, el arreglo de cinco ubicaciones y el ArrayList de pistas.
getNombre, getCodigoIdentificacion,	Ninguno	String	Devuelven el dato correspondiente del caso.

getDetectiveResponsable			
getUbicaciones	Ninguno	Ubicacion[]	Devuelve el arreglo de ubicaciones.
getPistas	Ninguno	ArrayList<Pista>	Devuelve la colección de pistas.
registrarUbicacion	int posicion, Ubicacion ubicacion	void	Registra una ubicación si la posición es válida, está libre y el objeto no es null.
consultarUbicacion	int posicion	Ubicacion	Devuelve la ubicación; genera una excepción si la posición es inválida o está vacía.
modificarUbicacion	int posicion, int nivelRiesgo, String estado	void	Modifica el riesgo y el estado de una ubicación existente.
descartarUbicacion	int posicion	void	Elimina una ubicación existente dejando su posición en null.
validarPosicion	int posicion	void	Genera IndexOutOfBoundsException si el índice está fuera del arreglo.
contarUbicaciones	Ninguno	int	Cuenta las posiciones que contienen una ubicación.

contarEspaciosDisponibles	Ninguno	int	Calcula cuántos espacios quedan libres.
obtenerUbicacionMayorRiesgo	Ninguno	Ubicacion	Devuelve la ubicación de mayor riesgo o null si no hay registros.
registrarPista	Pista nuevaPista	void	Agrega una pista; genera una excepción si es null o su código está repetido.
buscarPista	String codigo	Pista	Busca por código y devuelve la pista o null si no existe.
modificarPista	String codigo, String nuevaDescripcion, String nuevoTipoEvidencia, int nuevaImportancia, int nuevaConfiabilidad	void	Modifica una pista existente y valida sus niveles mediante los setters.
eliminarPista	String codigo	void	Elimina la pista indicada; genera una excepción si no existe.
obtenerPistaMayorImportancia	Ninguno	Pista	Devuelve la pista de mayor importancia o null si la colección está vacía.
obtenerPistaMayorConfiabilidad	Ninguno	Pista	Devuelve la pista de mayor confiabilidad o

			null si la colección está vacía.
calcularPromedioImportancia	Ninguno	double	Calcula el promedio de importancia; devuelve 0 si no hay pistas.

Clase: Ubicacion

Método	Parámetros	Retorno	Propósito
Ubicacion	String codigo, String nombre, String direccion, int nivelRiesgo, String estado	No aplica	Valida el riesgo y asigna los valores iniciales.
getCodigo, getNombre, getDireccion, getEstado	Ninguno	String	Devuelven el atributo correspondiente.
getNivelRiesgo	Ninguno	int	Devuelve el nivel de riesgo.
setCodigo, setNombre, setDireccion, setEstado	Un String con el valor correspondiente	void	Modifican el atributo correspondiente.
setNivelRiesgo	int nivelRiesgo	void	Valida y modifica el riesgo.
validarNivelRiesgo	int nivelRiesgo	void	Genera IllegalArgumentException si el riesgo no está entre 1 y 10.
toString	Ninguno	String	Devuelve los datos de la ubicación como texto.

Clase: Pista

Método	Parámetros	Retorno	Propósito
Pista	String codigo, String descripcion, String tipoEvidencia, int nivelImportancia, int nivelConfiabilidad	No aplica	Valida los niveles y asigna los valores iniciales.
getCodigo, getDescripcion, getTipoEvidencia	Ninguno	String	Devuelven el atributo correspondiente.
getNivelImportancia, getNivelConfiabilidad	Ninguno	int	Devuelven el nivel correspondiente.
setCodigo, setDescripcion, setTipoEvidencia	Un String con el valor correspondiente	void	Modifican el atributo correspondiente.
setNivelImportancia	int nivelImportancia	void	Valida y modifica la importancia.
setNivelConfiabilidad	int nivelConfiabilidad	void	Valida y modifica la confiabilidad.
validarNivelImportancia	int nivelImportancia	void	Genera IllegalArgumentException si la importancia no está entre 1 y 10.
validarNivelConfiabilidad	int nivelConfiabilidad	void	Genera IllegalArgumentException si la confiabilidad no está entre 0 y 100.

toString	Ninguno	String	Devuelve los datos de la pista como texto.

Clase: App o Main

Método	Parámetros	Retorno	Propósito
main	String[] args	void	Inicia el programa, controla el menú y maneja excepciones con try-catch-finally.
mostrarMenu	Ninguno	void	Muestra las trece opciones.
crearCaso	Scanner scanner	Caso	Solicita los datos y devuelve un nuevo caso.
registrarUbicacion	Scanner scanner, Caso caso	void	Solicita la posición y los datos para registrar una ubicación.
consultarUbicaciones	Caso caso	void	Muestra las ubicaciones registradas y sus posiciones.
consultarUbicacion	Scanner scanner, Caso caso	void	Solicita una posición y muestra su ubicación.
modificarUbicacion	Scanner scanner, Caso caso	void	Solicita la posición, el nuevo riesgo y el estado para actualizar una ubicación.
descartarUbicacion	Scanner scanner, Caso caso	void	Solicita la posición de la ubicación que se eliminará.
registrarPista	Scanner scanner, Caso caso	void	Solicita los datos y registra una pista.
consultarPistas	Caso caso	void	Muestra las pistas o informa que no hay registros.

buscarPistas	Scanner scanner, Caso caso	void	Solicita un código y muestra la pista encontrada.
modificarPista	Scanner scanner, Caso caso	void	Solicita los nuevos datos y valida ambos niveles antes de modificar la pista.
eliminarPista	Scanner scanner, Caso caso	void	Solicita el código y elimina la pista correspondiente.
mostrarReporte	Caso caso	void	Muestra los datos, cantidades, espacios disponibles, máximos y promedio del caso.

1.4 Uso del arregle básico y del ArrayList

Propiedad	Clase	Estructura	Objetos que almacena	Tamaño
ubicaciones	Caso	Ubicacion[]	Objetos de tipo Ubicacion.	Fijo: cinco posiciones.
pistas	Caso	ArrayList<Pista>	Objetos de tipo Pista.	Dinámico: cambia al agregar o eliminar pistas.

1.5 Verificación de posiciones vacías

El programa comprueba si ubicaciones[posicion] != null antes de utilizar una ubicación. Al consultar todas, omite las posiciones vacías; al consultar, modificar o descartar una específica, informa si no existe. Antes de acceder al arreglo, valida que el índice esté dentro de sus límites.

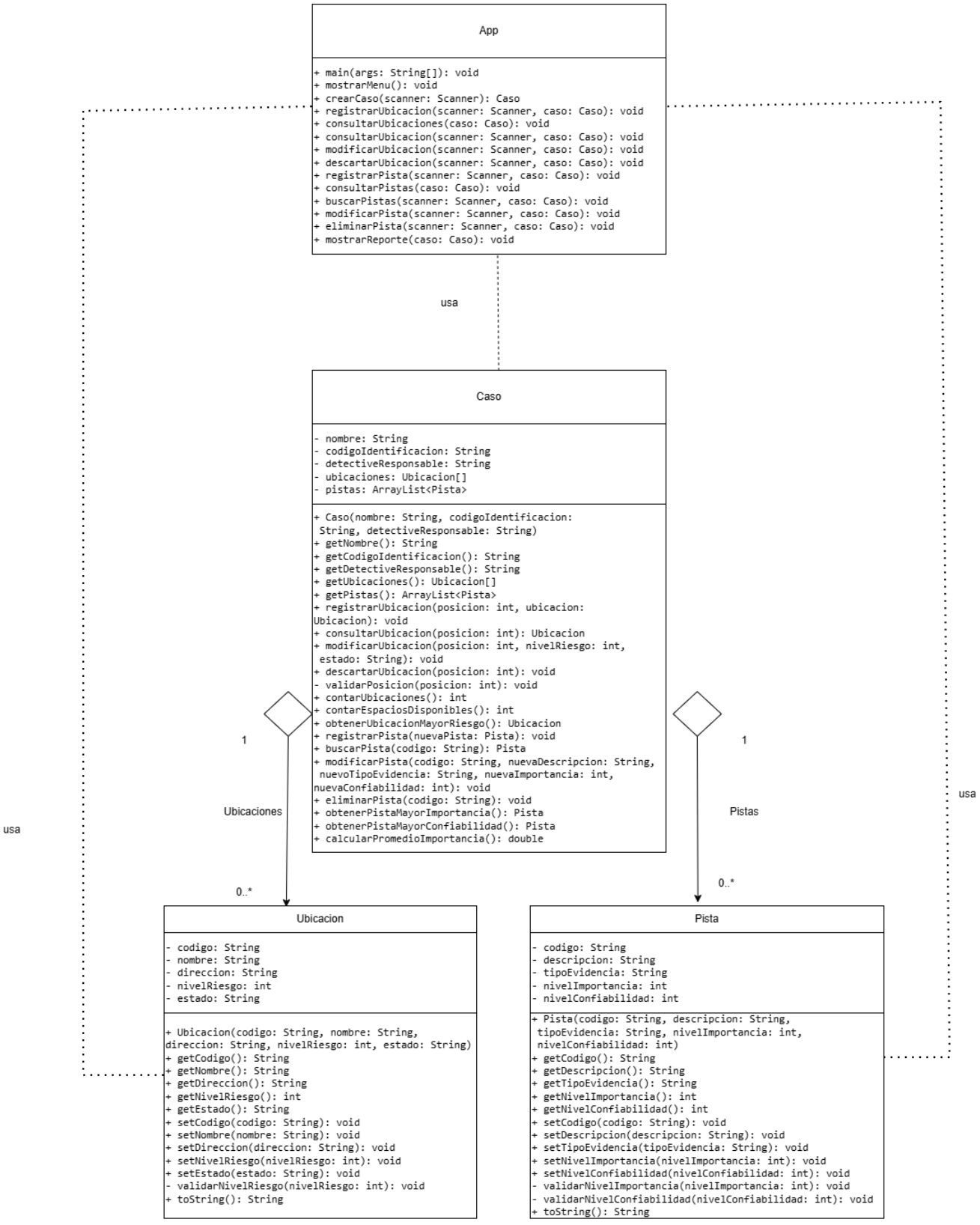
1.6 Manejo de excepciones

Excepción	Situación que la produce
InputMismatchException	El usuario ingresa texto o un decimal cuando se solicita un número entero.
IllegalArgumentException	Se ingresan niveles fuera de rango, códigos de pistas repetidos, objetos null o se intenta operar con registros inexistentes o posiciones ocupadas.
IndexOutOfBoundsException	Se intenta acceder a un índice fuera del arreglo de ubicaciones.

1.7 Manejo de excepciones

Estructura	Función en el programa
try	Contiene la lectura de la opción y el switch que ejecuta la operación seleccionada, donde pueden producirse excepciones.
catch	Captura las excepciones y muestra un mensaje de error. Ante una entrada numérica incorrecta, la limpia con scanner.nextLine().
finally	Muestra “Operación finalizada”, tanto si la operación fue exitosa como si ocurrió una excepción.
do-while	Repite el menú hasta que el usuario selecciona la opción 13. Salir.

Diagrama


App
+ main(args: String[]): void
+ mostrarMenu(): void
+ crearCaso(scanner: Scanner): Caso
+ registrarUbicacion(scanner: Scanner, caso: Caso): void
+ consultarUbicaciones(caso: Caso): void
+ consultarUbicacion(scanner: Scanner, caso: Caso): void
+ modificarUbicacion(scanner: Scanner, caso: Caso): void
+ descartarUbicacion(scanner: Scanner, caso: Caso): void
+ registrarPista(scanner: Scanner, caso: Caso): void
+ consultarPistas(caso: Caso): void
+ buscarPistas(scanner: Scanner, caso: Caso): void
+ modificarPista(scanner: Scanner, caso: Caso): void
+ eliminarPista(scanner: Scanner, caso: Caso): void
+ mostrarReporte(caso: Caso): void
usa
Caso
- nombre: String
- codigoIdentificacion: String
- detectiveResponsable: String
- ubicaciones: Ubicacion[]
- pistas: ArrayList<Pista>
+ Caso(nombre: String, codigoIdentificacion: String, detectiveResponsable: String)
+ getNombre(): String
+ getCodigoIdentificacion(): String
+ getDetectiveResponsable(): String
+ getUbicaciones(): Ubicacion[]
+ getPistas(): ArrayList<Pista>
+ registrarUbicacion(posicion: int, ubicacion: Ubicacion): void
+ consultarUbicacion(posicion: int): Ubicacion
+ modificarUbicacion(posicion: int, nivelRiesgo: int, estado: String): void
+ descartarUbicacion(posicion: int): void
- validarPosicion(posicion: int): void
+ contarUbicaciones(): int
+ contarEspaciosDisponibles(): int
+ obtenerUbicacionMayorRiesgo(): Ubicacion
+ registrarPista(nuevaPista: Pista): void
+ buscarPista(codigo: String): Pista
+ modificarPista(codigo: String, nuevaDescripcion: String, nuevoTipoEvidencia: String, nuevaImportancia: int, nuevaConfiabilidad: int): void
+ eliminarPista(codigo: String): void
+ obtenerPistaMayorImportancia(): Pista
+ obtenerPistaMayorConfiabilidad(): Pista
+ calcularPromedioImportancia(): double
1
Ubicaciones
0..*
1
Pistas
0..*
usa
usa
Ubicacion
- codigo: String
- nombre: String
- direccion: String
- nivelRiesgo: int
- estado: String
+ Ubicacion(codigo: String, nombre: String, direccion: String, nivelRiesgo: int, estado: String)
+ getCodigo(): String
+ getNombre(): String
+ getDireccion(): String
+ getNivelRiesgo(): int
+ getEstado(): String
+ setCodigo(codigo: String): void
+ setNombre(nombre: String): void
+ setDireccion(direccion: String): void
+ setNivelRiesgo(nivelRiesgo: int): void
+ setEstado(estado: String): void
- validarNivelRiesgo(nivelRiesgo: int): void
+ toString(): String
Pista
- codigo: String
- descripcion: String
- tipoEvidencia: String
- nivelImportancia: int
- nivelConfiabilidad: int
+ Pista(codigo: String, descripcion: String, tipoEvidencia: String, nivelImportancia: int, nivelConfiabilidad: int)
+ getCodigo(): String
+ getDescripcion(): String
+ getTipoEvidencia(): String
+ getNivelImportancia(): int
+ getNivelConfiabilidad(): int
+ setCodigo(codigo: String): void
+ setDescripcion(descripcion: String): void
+ setTipoEvidencia(tipoEvidencia: String): void
+ setNivelImportancia(nivelImportancia: int): void
+ setNivelConfiabilidad(nivelConfiabilidad: int): void
- validarNivelImportancia(nivelImportancia: int): void
- validarNivelConfiabilidad(nivelConfiabilidad: int): void
+ toString(): String